
Probabilistic Skip Connections for Deterministic Uncertainty Quantification in Deep Neural Networks

Felix Jimenez¹ Matthias Katzfuss¹

Abstract

Deterministic uncertainty quantification (UQ) in deep learning seeks to provide uncertainty estimates using a single pass through a neural network. Existing single-pass methods require feature extractors that are both *sensitive* and *smooth*, necessitating modifications to the training process or restricting applicability to regression tasks. We propose probabilistic skip connections (PSCs) as a drop-in component to provide single-pass UQ to pretrained networks without any modifications to the network or its training procedure. Our approach selects the internal layer for PSC placement based on measures of *neural collapse*. We support our design with an empirical analysis of the impact of spectral normalization on neural collapse and demonstrate that our construction effectively disentangles aleatoric and epistemic uncertainty. Finally, we show that PSCs can match or surpass existing single-pass methods that modify network training in terms of both uncertainty quantification and out-of-distribution detection (OOD). By retrofitting existing models, PSCs enable high-quality UQ and OOD capabilities without requiring changes to the underlying network.

1. Introduction

Uncertainty quantification (UQ) for deep learning has traditionally relied on neural network ensembles [11] or Bayesian methods that sample network weights with techniques like Markov Chain Monte Carlo (MCMC) [17], Monte Carlo dropout [5] or methods approximating the posterior for network weights [15, 20]. These methods involve either training the network multiple times or performing several passes through a single network, both of which are computationally intensive.

To address this burden, single-pass deterministic UQ meth-

ods have emerged as a promising solution [13, 16, 23] and estimate uncertainty with one forward pass through the network by using the network’s feature maps. However, if the network is experiencing *feature collapse*, a phenomenon where unique inputs are mapped to identical feature maps, then such methods will not work. Therefore, methods using feature maps rely on well structured feature spaces and distance-aware output layers, requiring modified network training and a particular network architecture.

An alternative approach is to use the outputs of intermediate layers where feature collapse has not occurred [9], but this work has been limited to regression. As of yet, there has not been a single-pass deterministic uncertainty method that can be dropped into an existing classification neural network that does not require changing the network or its training procedure.

In this work, we propose probabilistic skip connections (PSCs), which allow for deterministic UQ by identifying the intermediate layers of a neural network that generate feature maps that can be used with existing deterministic UQ methods. We find these layers by measuring neural collapse, a phenomenon closely related to but distinct from feature collapse, and show that our procedure allows for deterministic UQ in the presence of feature collapse, even for networks for which existing deterministic UQ methods were not applicable. An overview of our approach is shown in Figure Fig. 1.

Our contributions can be summarized as follows:

- We propose PSCs to allow for deterministic UQ for classification networks without needing to retrain the feature extractor with spectral normalization.
- Through an empirical study, we motivate our constructions of PSCs by examining the impact of spectral normalization on intermediate neural collapse.
- We show our approach rivals and often surpasses deterministic UQ methods requiring training modifications.
- We demonstrate that deterministic UQ methods extend to a larger family of neural networks than previously suggested.

¹Department of Statistics, University of Wisconsin–Madison. Correspondence to: Felix Jimenez <felix.m.jimenez3@gmail.com>.

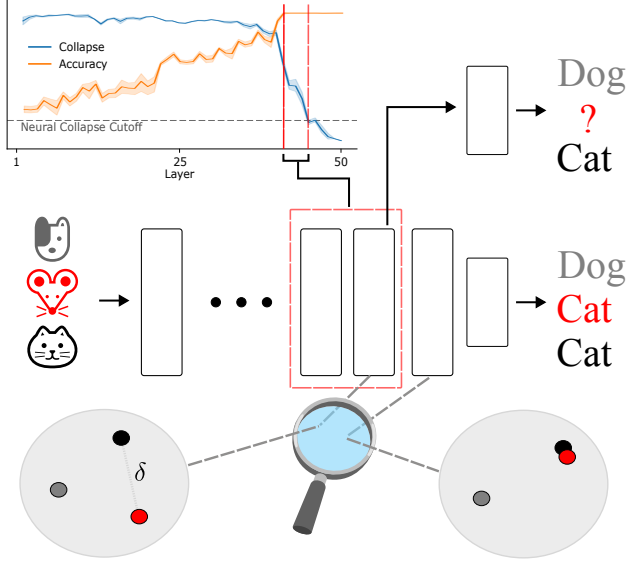


Figure 1. Neural collapse metrics help identify intermediate layers with low feature collapse and high nearest-centroid accuracy, allowing us to place probabilistic skip connections (PSCs) that enhance uncertainty quantification. The top left shows the trade-off between feature collapse and accuracy across network depth. In this example, dog, mouse, and cat images are processed through a network trained only on dog and cat images, with the mouse being out-of-distribution (OOD). Without a PSC, the network incorrectly labels the mouse as “Cat”. However, adding a PSC at a layer with high accuracy and low collapse enables the model to better express uncertainty for the mouse image. The magnifying glass view illustrates why: in layers identified through neural collapse metrics, the feature vector difference, δ , between the cat and mouse remains large. By the penultimate layer, this difference collapses, leading to overconfidence. Identifying an appropriate intermediate layer maintains feature sensitivity while preserving predictive performance.

2. Background

In this section, we provide a brief review of UQ, cover essential topics from feature geometry in deep learning, and discuss how feature collapse presents a challenge to performing deterministic UQ for deep learning.

Notation and terminology for the remainder of the paper is described below. This work assumes neural networks of the following form, $f(\mathbf{x}) = g \circ h(\mathbf{x})$, where h is referred to as the feature extractor and g is the output layer. The function h is assumed to be the composition of L layers, $h(\mathbf{x}) = h_L \circ \dots \circ h_1(\mathbf{x})$, and we denote the image of h as $\mathcal{H}$, which we refer to as the *feature space*. Then g is taken to be a model mapping the feature space to logit space.

For input $\mathbf{x} \in \mathcal{X}$, hidden layer $j \in \{1, \dots, L\}$, we refer to $h^{(j)}(\mathbf{x}) := h_j \circ \dots \circ h_1(\mathbf{x})$ as an intermediate representation of $\mathbf{x}$. The final intermediate representation, $h(\mathbf{x}) = h_L \circ$

$\dots \circ h_1(\mathbf{x})$, is referred to as the feature map of $\mathbf{x}$.

Let $d_{\mathcal{X}}$ denote an appropriate distance metric on the data manifold, such that $d_{\mathcal{X}}(\mathbf{x}_1, \mathbf{x}_2) > 0$ implies that $\mathbf{x}_1, \mathbf{x}_2 \in \mathcal{X}$ are semantically distinct. We abbreviate in-distribution as iD and out-of-distribution as OOD.

The *spectral norm* of a matrix, $\mathbf{A} \in \mathbb{R}^{n \times m}$, is given by $\sigma_{\max}(\mathbf{A})$, where $\sigma_{\max}$ denotes the matrix’s largest singular value.

2.1. Uncertainty quantification

We now introduce the framework we use for UQ in supervised learning. Consider a conditional distribution $p(y|\mathbf{x})$, where $\mathbf{x} \in \mathcal{X} \subset \mathbb{R}^d$ represents inputs, and $y \in \mathcal{Y}$ represents corresponding responses. Given a dataset $\mathcal{D} = \{\mathbf{x}_i, y_i\}$, we learn a parametric function $f_{\theta} : \mathcal{X} \rightarrow \mathcal{Y}$ by minimizing the empirical risk $\sum_{i=1}^n \mathcal{L}(f_{\theta}(\mathbf{x}_i), y_i)$.

For a new input, $\mathbf{x}^*$, we use the tuned model to generate a prediction, $f_{\theta^*}(\mathbf{x}^*)$, for the true response, $y^* \sim p(y|\mathbf{x}^*)$. The uncertainty in our prediction can be divided into two components: aleatoric and epistemic uncertainty.

Epistemic uncertainty arises from a lack of data sufficiently similar to $\mathbf{x}^*$, resulting in a lack of knowledge necessary for f to approximate $p(y|\mathbf{x})$ at $\mathbf{x}^*$. Epistemic uncertainty can be reduced by gathering more data near $\mathbf{x}^*$, so it is often referred to as *model uncertainty*.

Aleatoric uncertainty refers to irreducible error, capturing the inherent ambiguity in the relationship between $\mathbf{x}$ and y . For example, we may know a scale is accurate up to 0.5 g after calibrating against reference materials, but even with that information, a new measurement can only estimate the true weight to within 0.5 g. While collecting more samples helps quantify measurement error, the accuracy of any single measurement is limited by the scale itself.

Scoring rules are the gold standard for assessing probabilistic forecasts. Consider a sample space Ω and family of probabilistic forecasts $\mathcal{P}$. A scoring rule, S , is a function $S : \Omega \times \mathcal{P} \rightarrow \overline{\mathbb{R}}$, where $\overline{\mathbb{R}} = [-\infty, \infty]$, that measures the quality of predictions made by $P \in \mathcal{P}$ when data was generated from a distribution $Q \in \mathcal{P}$. Letting $S(P, Q) = \mathbb{E}_{y \sim Q}[S(P, y)]$, scoring rules such that $S(P, Q) \geq S(Q, Q)$ with equality holding iff $P = Q$ are said to be strictly proper [7, 8]. Strictly proper scoring rules are the gold standard for comparing probabilistic forecasts and include the common negative log-likelihood and Brier score. To make practical use of scoring rules, we use sampled-based estimates $\frac{1}{n} \sum_{i=1}^n S(P, y_i)$.

UQ for neural networks traditionally relies on neural network ensembles [11] or Bayesian methods that sample network weights with techniques like Markov Chain Monte Carlo (MCMC) [17], Monte Carlo dropout [5] or approx-

imate the posterior of network weights [2, 15, 20]. These methods involve either training the network multiple times or performing several passes through a single network, both of which are computationally intensive.

Deterministic UQ for neural networks refers to single-forward-pass deterministic methods [13, 16, 23], which estimate uncertainty with one forward pass through the network by using the network’s feature maps. The use of feature maps relies on well structured feature spaces and distance aware output layers. In the context of well structure feature spaces, Liu et al. [13] argue that reliable predictive uncertainty estimation requires the network’s feature extractor, h , to be *distance preserving*, which they, like others [23], equate to a bi-Lipschitz constraint:

$$L_1 d_{\mathcal{X}}(\mathbf{x}_1, \mathbf{x}_2) \leq \|h(\mathbf{x}_1) - h(\mathbf{x}_2)\|_H \leq L_2 d_{\mathcal{X}}(\mathbf{x}_1, \mathbf{x}_2). \quad (1)$$

Simply put, meaningful changes in the input space should produce corresponding changes in the feature space, and these changes’ magnitudes should be preserved. To encourage a bi-Lipschitz continuous feature extractor, training is modified to bound the spectral norm of the weight matrices of individual layers, $\sigma_{\max}(\mathbf{A}) < C, C \in \mathbb{R}^+$. If training is not modified, then the network is likely to experience *feature collapse*, where unique inputs are mapped to identical feature maps. If there is feature collapse, then UQ will suffer, because the network will be unable to differentiate between points similar to and points different than the training data.

Assuming a well structured feature space, we also need a distance-aware output layer. The spectral normalized Gaussian process (SNGP) [13] uses a Gaussian process as the output layer, while deep deterministic uncertainty (DDU) [16] uses Gaussian discriminate analysis for quantifying epistemic uncertainty and uses the network’s predictive entropy for quantifying aleatoric uncertainty. Deterministic uncertainty quantification (DUQ) [23] uses RBF networks.

An alternative approach to remedying feature collapse involves ensembling intermediate representations [9]. The idea is that, while the feature maps may be collapsed, there may be an intermediate representation without collapse, so by ensembling over layers one can effectively get a distance-preserving feature map. However, this work has only been shown to work on regression tasks.

2.2. Feature geometry

We now discuss the geometry of feature space, $\mathcal{H}$, from two perspectives: neural collapse and feature collapse.

Neural Collapse [18] is a phenomenon observed during the *terminal phase of training* (TPT), when training error has saturated but training loss continues to decrease, and is characterized by a consistent structure in both the linear

classifier, g , and the feature space, $\mathcal{H}$. Neural collapse is characterized by the following criteria:

- $\mathcal{NC1}$: Vanishing within-class variance of activations.
- $\mathcal{NC2}$: Centered classwise activation means have equal length, equal angle between them and are pairwise maximally spread.
- $\mathcal{NC3}$: Up to scaling, class-wise activation means and the weights of g converge.
- $\mathcal{NC4}$: g converges to a nearest-class-center classifier.

While neural collapse was initially observed at the penultimate layer, intermediate neural collapse has been shown to occur [19] as well.

Feature Collapse describes unique inputs being mapped to identical feature maps. The *smoothness* of a neural network describes the degree of continuity in the mapping from input to feature space, while the *sensitivity* of the network describes how the network’s features respond to perturbations in the inputs. A network that is both smooth and sensitive will satisfy a bi-Lipschitz constraint as in Eq. (1), but a network with feature collapse will not satisfy this constraint.

Feature geometry and deterministic UQ are closely related, as the bi-Lipschitz requirement imposes constraints on the geometry of the feature space. However, little work has been done to leverage results from neural collapse to assess feature collapse in a network, and thus evaluate the effectiveness of deterministic UQ methods without relying on spectral normalization.

For example, relevant to us, $\mathcal{NC1}$ characterizes the network’s propensity to map inputs from the same class to similar, if not identical, feature maps. While $\mathcal{NC4}$ characterizes the separability of different classes at a particular layer. Together these two metrics share a seemingly similar role to the sensitivity and smoothness of a network. In Sec. 4 we use this perspective to directly relate feature collapse and neural collapse.

3. Related work

We now contextualize our work by contrasting it with deterministic deep UQ methods that directly address feature collapse. In Tab. 1, we compare our work to existing methods to highlight that our approach is the first classification-compatible method to allow for deep deterministic UQ without modifying training.

Additionally, the methods in Tab. 1 that work for classification are limited to particular network structures. Both SNGP and DDU are limited to networks with residual layers and DUQ was built with RBF networks [12] in mind.

Method	F.C. Soln.	Retrain	Task
SNGP [13], DDU [16]	SN	Yes	Cl.
DUQ [23]	Jacob.	Yes	Cl.
DVE [9]	Skip	No	Reg.
Ours	Skip	No	Cl.

Table 1. **Our method enables deterministic UQ for classification without retraining**, which had previously been limited to regression tasks.

However, our approach does not require residual layers to be present and, as shown in Sec. 5, we are able to augment VGG16 to perform as well as Resnet18 trained with spectral normalization. Our work demonstrates that deterministic UQ methods apply to a broader range of networks than previously suggested.

While the deep Vecchia ensemble (DVE) [9] also works for networks without residual layers, our work applies to classification while DVE is only relevant to regression. Additionally, our approach of selecting layers using measures of neural collapse is more streamlined than DVE, which ensembles models built on all the intermediate layers.

4. Probabilistic skip connections (PSCs)

The constraint in Eq. (1) is typically enforced by modifying network training; however, we propose instead to identify an intermediate layer $h^{(j)}$ for some $j \in \{1, \dots, L\}$ that already meets the bi-Lipschitz requirement in Eq. (1).

Assuming such a subnetwork exists, we define a new model $f'(\mathbf{x}) := g' \circ t \circ h^{(j)}(\mathbf{x})$, where t maps the intermediate representations of $h^{(j)}$ to a lower-dimensional feature space and g' is a new output layer. Notice that g' can be chosen from any existing deterministic UQ methods, such as a Gaussian process (as in SNGP), a Gaussian mixture model or linear model (as in DDU), or other distance-aware models as defined in Liu et al. [13].

Our workflow for finding the layer j , getting the projection function t and finally using the new distance aware g' is illustrated in Fig. 2. The left most panel of Fig. 2 shows the identification of the layer using *neural collapse*, which is discussed in Sec. 4.1. The center panel represents the function t , as discussed in Sec. 4.2. Finally, the right most panel shows the function g' (discussed in Sec. 4.3) being used to determine that a sample is iD and then subsequently making a prediction for that input.

4.1. Choosing layer: collapse-accuracy trade-off

The leftmost panel of Fig. 2 shows accuracy increasing with layer depth and collapse creeping towards a cutoff value. Below we make it clear what exactly is being computed for

collapse and accuracy.

Collapse at layer l is quantified by considering an approximation of $\mathcal{NC1}$ from Rangamani et al. [19] and is denoted $\mathcal{NC1}$ for the remainder of the paper. To begin, for layer l and class c we compute the class-wise feature means and the grand feature mean,

$$\mu_c^l = \frac{1}{N} \sum_{i \in \mathcal{I}_c} h^{(l)}(\mathbf{x}_i), \quad \mu_G^l = \frac{1}{C} \sum_{c=1}^C \mu_c^l$$

where $\mathcal{I}_c \subset \mathcal{I} = \{1, \dots, N\}$, denote the instances in class c . We then compute the within class and total covariance about these means,

$$\Sigma_W^l = \frac{1}{NC} \sum_{c=1}^C \sum_{i \in \mathcal{I}_c}^N (h^{(l)}(\mathbf{x}_i) - \mu_c^l)(h^{(l)}(\mathbf{x}_i) - \mu_c^l)^T$$

$$\Sigma_T^l = \frac{1}{NC} \sum_{c=1}^C \sum_{i \in \mathcal{I}_c}^N (h^{(l)}(\mathbf{x}_i) - \mu_G^l)(h^{(l)}(\mathbf{x}_i) - \mu_G^l)^T.$$

Finally, we can use the within and total covariance to compute the approximation $\mathcal{NC1} \approx \text{Tr}(\Sigma_W)/\text{Tr}(\Sigma_T)$. A layer exhibits *feature variability suppression* if $\text{Tr}(\Sigma_W)/\text{Tr}(\Sigma_T) < \epsilon = 0.2$. We use this feature variability suppression as a proxy for feature collapse and in our figures we refer to this cutoff as the “Neural Collapse Cutoff” for simplicity.

Accuracy of layer l is quantified by measuring the accuracy of a nearest centroid classifier (NCC) that uses $h^{(l)}(\mathbf{x})$ to classify observations and is denoted by $\mathcal{NC4}$ for the remainder of the paper. In this work we use the NCC accuracy of the validation set.

Trading off collapse and accuracy is done by finding the layer with the highest possible NCC accuracy such that $\mathcal{NC1} > 0.2$, taking the layer before and after collapse if the nearest layer is especially close to the neural collapse cutoff. We refer to the chosen layer(s) as the *candidate layer(s)*.

In Fig. 2 the candidate layers have a dashed red box around them in the leftmost panel. In Sec. 5.1, we carry out experiments to justify using $\mathcal{NC1}$ and $\mathcal{NC4}$ to select the candidate layer.

4.2. Processing intermediate Representations

Once we have found the candidate layers we need to combine them, if there are multiple, and project to a lower dimensional shape, if they are high dimensional.

Combining layers involves reshaping them and concatenating. For a convolutional layers $h^{(j)}(\mathbf{x}) \in \mathbb{R}^{C \times h \times w}$ and $h^{(j+1)}(\mathbf{x}) \in \mathbb{R}^{C \times h' \times w'}$ we would reshape the layers to $\mathbb{R}^{C \times hd}$ and $\mathbb{R}^{C \times h'w'}$, respectively. We then concatenate

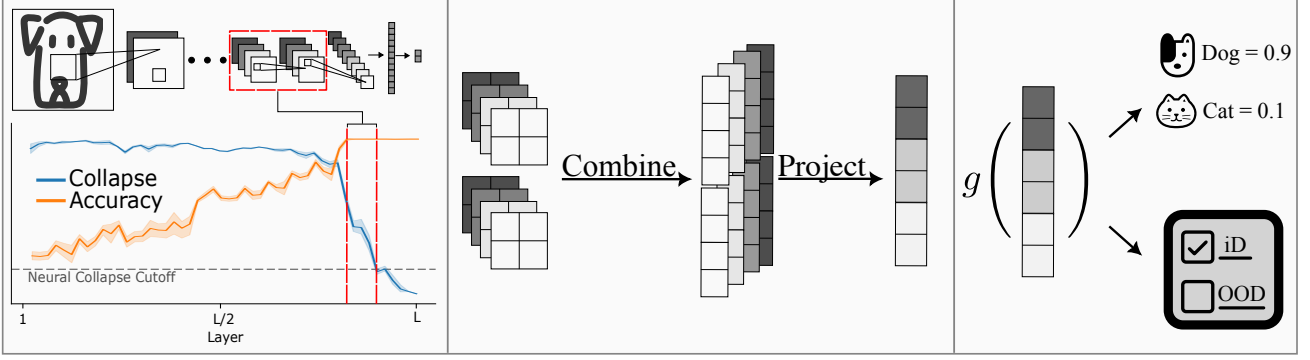


Figure 2. Adding PSCs to a pretrained model involves measuring collapse (left) projecting intermediate layer to feature vector (center) fitting probabilistic model to projected layer (right). The first step is to find a subset of the layers that tradeoff accuracy and collapse, higher is better for both accuracy and collapse. Once we have those layers we then combine them and project to a single feature vector. Finally, we fit a probabilistic model to that feature vector which can identify iD vs OOD and make class predictions.

channel wise to get a combined input of shape $\mathbb{R}^{C \times h'w' + hw}$, which we will refer to as the candidate layer as the layers have now been combined. For simplicity, we will let $\mathbf{X}$ denote candidate layer and assume $\mathbf{X} \in \mathbb{R}^{C \times hd}$.

Projecting the layers is done by first computing the channelwise covariance matrices for the candidate layer, which results in a tensor of covariances $\Sigma_c \in \mathbb{R}^{C \times hw \times hw}$. We then perform a Tucker decomposition [22], that is we approximate the covariance using,

$$\Sigma_c \approx \mathcal{G} \times_1 \mathbf{A} \times_2 \mathbf{B} \times_3 \mathbf{C},$$

where $\times_i$ denotes the i -mode product. The matrices $\mathbf{A} \in \mathbb{R}^{C \times c_{proj}}$, $\mathbf{B} \in \mathbb{R}^{hw \times d_{proj}}$ and $\mathbf{C} \in \mathbb{R}^{hw \times d_{proj}}$ are the factor matrices and can be thought of as the principal components in each mode. The tensor $\mathcal{G} \in \mathbb{R}^{c_{proj} \times d_{proj} \times d_{proj}}$ is called the core tensor and its entries shows the level of interaction between the different components. Given the Tucker decomposition of Σ_c we can then project $\mathbf{X}$ onto these factor matrices to get a low dimensional representation $\mathbf{Z}$,

$$\mathbf{Z} = \mathbf{X} \times_1 \mathbf{A}_{:,0:c_{proj}} \times_2 \mathbf{B}_{:,0:d_{proj}}.$$

Where $\mathbf{Z} \in \mathbb{R}^{c_{proj} \times d_{proj}}$ and $\mathbf{K}_{:,0:d}$ denotes the first d columns of the matrix $\mathbf{K}$. Finally, we can flatten this projection to get the final feature vector $\mathbf{Z} \in \mathbb{R}^{c_{proj}d_{proj}}$.

The primary benefit of the Tucker decomposition is it allows us to choose how much we reduce the channel dimension versus how much we reduce the spatial dimension. This means we must choose two parameters at this stage: the projection dimension of the channels (c_{proj}) and the dimension of the spatial projections (d_{proj}). In Section Sec. 5.3, we demonstrate the low-dimensional nature of intermediate representations and provide guidance on setting these parameters based on an empirical study. In short, we recom-

mend setting c_{proj} and d_{proj} to be the smallest values such that neither $\mathcal{NC}1$ and $\mathcal{NC}4$ change dramatically.

4.3. Linear layers as PSCs

After processing the outputs of an intermediate layer, we now have input-output pairs, $\{\mathbf{Z}_i, y_i\}_{i=1}^N$ with $\mathbf{Z}_i \in \mathbb{R}^{c_{proj}d_{proj}}$ being the output of the “Project” step in Fig. 2 and $y_i \in \{1, \dots, C\}$ being the class label. At this point any distance aware model can be used to make predictions.

In this work, we decompose g into two components, similar to the approach in DDU [16]. For iD/OOD classification, we use quadratic discriminant analysis, and for iD UQ, we apply a linear model with a Laplace approximation for the posterior of the coefficients. To ensure scalability, we employ the Kronecker-factored Laplace approximation (KFAC) [20] as implemented in the Laplace package in PyTorch [3]. However, our linear model and feature density could be substituted with any other probabilistic model; our primary contribution is demonstrating how deterministic UQ can be achieved by leveraging intermediate network layers.

5. Experiments

We now present experiments to show (1) $\mathcal{NC}1$ and $\mathcal{NC}2$ are good proxies for sensitivity and smoothness Sec. 5.1 (2) the PSC in Sec. 4.3 can rely on a well structured feature space and still work and Sec. 5.2 and (3) projecting as described in Sec. 4.2 does not hurt sensitivity and smoothness Sec. 5.3. For additional experiments see Appendix C.

5.1. Trade-off between sensitivity and smoothness

In Sec. 4, we proposed placing a PSC at an intermediate layer that is both sensitive and smooth. To find this layer, we find that $\mathcal{NC}1$ and $\mathcal{NC}4$ act as good proxies for sensitivity and smoothness, respectively.

To see why, recall that sensitivity ensures that distances in the hidden space correspond to meaningful distances in the data manifold, while $\mathcal{NC1}$ measures the within-class variance relative to the total variance. When $\mathcal{NC1} > \epsilon$, then the average within-class variance is non-zero, so the class-wise intermediate representations of that layer have not collapsed to a single point. At the same time, the value of $\mathcal{NC1}$ is decreasing as we move deeper into the network, so the network is more sensitive to perturbations that change the class label than it is to perturbations that keep the input within the same class. This ranking of the perturbations on the inputs is heuristically similar to having a lower Lipschitz bound on the network outputs at that layer.

For $\mathcal{NC4}$, we are measuring the accuracy of a nearest-centroid classifier, which will have high accuracy when semantically similar inputs are mapped to nearby points in feature space. This requires that the mapping from inputs to feature space is only sensitive to perturbations that are relevant for determining the class label, which is heuristically similar to having an upper Lipschitz bound on the network outputs.

To verify our intuition, we train a Resnet50 on CIFAR10 with and without spectral normalization. We compute both $\mathcal{NC1}$ and $\mathcal{NC4}$ on the outputs of each convolutional layer, the final average pooling layer and the output of the linear classifier using the implementation of Rangamani et al. [19]. The results of this are shown in Fig. 3, where the left panel shows the value of $\mathcal{NC1}$ vs layer and the right shows $\mathcal{NC4}$ vs layer.

Without spectral normalization, we see that $\mathcal{NC1}$ falls below the threshold for neural collapse conjectured in Rangamani et al. [19] many layers before the final layer. However, with spectral normalization the collapse only happens on the final layer. This suggests that spectral normalization has made the penultimate layer sensitive, which is the stated goal of using spectral normalization. In the right panel, we see that while the NCC accuracy with SN is below the final classifier accuracy for nearly every layer, by the end the NCC classifier matches the final classifier. This suggests that the network is still smooth, which again is one of the goals of using spectral normalization.

The performance of NCC for intermediate layers was observed by Ben-Shaul and Dekel [1], Galanti et al. [6], Rangamani et al. [19] with Galanti et al. [6] stating that this suggests the network has an “effective depth” that is lower than the nominal depth of the network. From that perspective, it seems that spectral normalization increases the effective depth of the network, which may have its own benefits outside of UQ, such as improving generalization [6] or supervised pretraining.

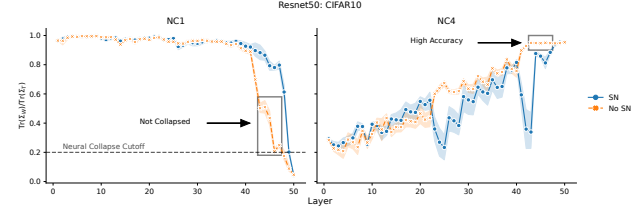


Figure 3. Spectral normalization (SN) pushes neural collapse to the end of the network but without SN there are intermediate layers that behave like the final layer of a network with SN. (Left) Using spectral normalization pushes the point where $\mathcal{NC1}$ goes below 0.2 to the final layer; without spectral normalization, there is a region where we have not yet collapsed but the value of $\mathcal{NC1}$ is different than that of with spectral norm. (Right) Using spectral norm decreases the performance of NCC for all but the last two layers, while using no spectral norm gives a larger number of layers whose NCC accuracy is nearly the same as the network’s end prediction. (Both Panels) The red boxes represent layers that have not yet collapsed but still maintain high NCC accuracy. This suggests that, even without spectral normalization, some layers are both *sensitive* and *smooth*.

5.2. DDU case study

In this experiment we take our PSC to be a version of DDU [16] to show using an intermediate layer without spectral normalization does just as well as using the feature space after training with spectral normalization. We also show that intermediate layers work even when there are no residual connections in the network.

Our experiment closely follows the MNIST experiment from Mukhoti et al. [16] which uses the Dirty-MNIST dataset for out-of-distribution detection. The in-distribution dataset is Dirty-MNIST, which is composed of MNIST and Ambiguous-MNIST, where Ambiguous-MNIST contains examples with high aleatoric uncertainty. The out-of-distribution dataset is taken to be FashionMNIST [24].

To start we train a network with and without spectral normalization on the Dirty-MNIST dataset. For the network with spectral normalization we use the same construction of DDU as in Mukhoti et al. [16]. For the network without spectral normalization we follow the procedure outlined in Sec. 4 with a single modification; we use quadratic discriminant analysis on the intermediate representation that feeds into our new linear output layer but we first use PCA to project this input to a lower dimensional space.

We can Distinguish Epistemic and Aleatoric Uncertainty Using Intermediate Layers: Fig. 4 panel (a) shows the feature densities of the different dataset for a Resnet-18 fit to Dirty-MNIST. The left most panel corresponds to using the original network, the center column corresponds to the network trained with spectral normalization and the final column corresponds to the original network augmented

Model	AUROC ($\uparrow$)	
	iD/OOD	iD/iD-Ambig.
Resnet-18	0.99	0.97
Resnet-18+SN	0.99	0.84
Resnet-18+PSC	0.99	0.93
VGG-16	0.93	0.98
VGG-16+SN	N/A	N/A
VGG-16+PSC	1.0	0.95

Table 2. **PSC AUROC matches spectral normalization and applies to more networks.** The AUROC for Dirty-MNIST vs. FashionMNIST (iD/OOD) and MNIST vs. Ambiguous-MNIST (iD/iD-Ambig.) for different models. Models below solid line do not have residual connections, so SN is not applicable.

with a PSC. We see that the feature density when using the intermediate layer distinguishes between iD and OOD just as well as when using spectral normalization, with both providing a modest improvement over the original network. In Appendix C.2 we demonstrate that predictive entropy of the PSC has similar properties to when using spectral normalization.

In Tab. 2, we present the AUROC values when using feature space density to compare iD and OOD examples, and we also present the AUROC values when using predictive entropy to compare iD-ambiguous to iD-clean. For both tasks, we see the AUROC of PSC matching the AUROC found when using spectral normalization or exceeding it, which confirms that using the intermediate space is a good substitute for spectral normalization.

Together, Fig. 4 and Tab. 2 demonstrate that feature density captures epistemic uncertainty but not aleatoric uncertainty, whereas predictive entropy captures aleatoric uncertainty. This behavior, observed in Mukhoti et al. [16], is replicated here; however, we achieve it solely using intermediate layers without relying on spectral normalization.

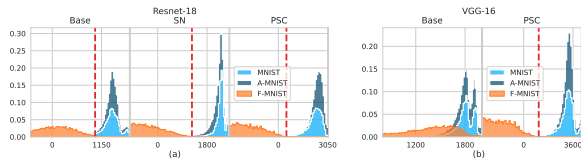


Figure 4. (a) **PSC feature density separates iD and OOD as well as using spectral normalization.** The base network’s feature density (left) can be improved equally well by using spectral normalization (middle) or an intermediate layer (right). (b) **Intermediate feature density also works for networks without residual connections and no spectral normalization.** The feature density of VGG-16 (left) conflates iD and OOD but using an intermediate layer (right) separates iD and OOD.

Model	Dataset	$\mathcal{NC1}$		$\mathcal{NC4}$	
		Orig.	Proj.	Orig.	Proj.
Resnet-18	MNIST	0.28	0.28	0.99	0.99
WRN	CIFAR10	0.73	0.75	0.95	0.95
Resnet50	CIFAR10	0.28	0.50	0.95	0.94
VGG-16	MNIST	0.26	0.36	0.97	0.97

Table 3. **Projecting intermediate representations to low-dimensional space does not induce collapse ($\mathcal{NC1} < 0.2$) or reduce NCC accuracy ($\mathcal{NC4}$).** Neural collapse metrics before (Orig.) and after projecting (Proj.) for a variety of networks and datasets.

DDU Works Without Residual Connections: In Fig. 4 panel (b) we replace Resnet-18 with VGG-16 [21], which has no residual connections, and rerun the experiment. We see that the original network has a poorly structured feature space as the feature density for iD and OOD instances overlap, but the intermediate feature space is surprisingly well behaved and appears as well separated as Resnet-18 trained with spectral normalization. This is confirmed by examining the final rows of Tab. 2, where the AUROC of VGG-16+PSC matches the best models in separating iD and OOD. Likewise, the predictive entropy of the PSC can be effectively used to when separating Ambiguous-MNIST and MNIST. Again, there are no residual connections in this model, yet we observe a deterministic method matching a network with a well regularized feature space.

5.3. Intermediate representations have exploitable low-rank structure

Fig. 3 shows that an intermediate layer of the network can have relatively little collapse while cleanly separating the classes in the dataset; however, these intermediate representations are often very high dimensional. This experiment demonstrates that using the Tucker decomposition as in Sec. 4.2 to project intermediate representations preserves the *smoothness* and *sensitivity* properties we observed at the *candidate layer* in the Sec. 5.1.

Tab. 3 shows the values of $\mathcal{NC1}$ and $\mathcal{NC4}$ at the candidate layer of networks before and after projecting. We see that projecting does not cause any collapse ($\mathcal{NC1} < 0.2$) for any of the networks or datasets. Additionally, we see little change in the nearest centroid accuracy as a result of projecting. This suggests that the smoothness and sensitivity of the intermediate layers is preserved after projecting.

It is important to note the results shown in Tab. 3 depend on the values of c_{proj} and d_{proj} as described in Sec. 4.2. Fig. 5 shows the impact of varying d_{proj} and c_{proj} on the neural collapse metrics for Resnet-18 on MNIST, similar behavior holds for other networks and datasets. We observe there are

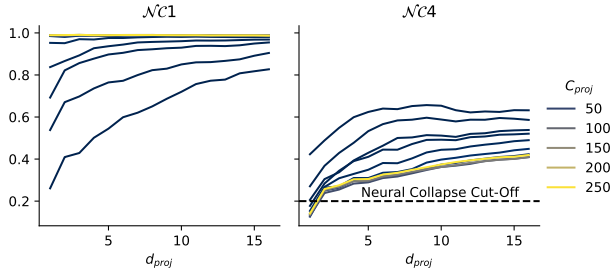


Figure 5. There are values of c_{proj} and d_{proj} that largely leave neural collapse metrics unchanged, but poor choices lead to collapse or low NCC accuracy. Neural collapse metrics for Resnet-18 on MNIST are shown as a function of c_{proj} and d_{proj} .

Method	Requires Retraining	Accuracy ($\uparrow$)	NLL ($\downarrow$)	ECE ($\downarrow$)
DNN	N	96.00 ± 0.01	0.15 ± 0.001	0.023 ± 0.001
DNN-PSC (Ours)	N	95.80 ± 0.01	0.136 ± 0.001	0.005 ± 0.000
DNN-SN	Y	96.00 ± 0.03	0.153 ± 0.002	0.022 ± 0.001

Table 4. PSC UQ performance on CIFAR10 matches networks trained using spectral normalization for Wide-ResNet-28-10. The mean and standard error of the mean are computed over ten seeds.

values of c_{proj} and d_{proj} where the previously uncollapsed layer, $\mathcal{NC}1 > \epsilon_c$, is now collapsed, $\mathcal{NC}1 < \epsilon_c$. However, for a large portion of values, there is both no collapse and the NCC accuracy is high. We find it sufficient to measure these two neural collapse metrics when setting d_{proj} and c_{proj} .

5.4. iD UQ performance for PSCs matches spectral normalization

We compare the iD UQ performance of PSCs to both the base network and a network trained with spectral normalization. In this experiment, we train a ResNet28x10 on the CIFAR-10 dataset, both with and without spectral normalization.

The results, presented in Tab. 4, show the mean and standard error of the mean over ten random seeds. While the original network achieves the highest accuracy, PSCs provide performance that matches or exceeds spectral normalization across all UQ metrics, with a notable improvement in expected calibration error (ECE). Additional results are available in Appendix C.

6. Conclusion

In this paper, we introduce probabilistic skip connections (PSCs), which add a distance-aware skip connection at an intermediate layer of a neural network. This enables deterministic UQ methods to overcome feature collapse without requiring the neural network to be retrained with spectral

normalization. We use measures of neural collapse to determine the appropriate layer for placing the PSCs and argue that these metrics serve as effective proxies for feature collapse. We support this claim by demonstrating through experiments that spectral normalization during training shifts the layers where neural collapse metrics stabilize toward the end of the network, effectively increasing the network’s depth. In contrast, our skip connection approach truncates the network at its initial effective depth. We validate this by showing that a modified version of Mukhoti et al. [16]’s deterministic uncertainty quantification (DDU) performs equally well with spectral normalization and with our PSCs. Additionally, we investigate the sensitivity of our method to changes in the parameters governing the projected dimensions of intermediate features and provide a comparative analysis.

While our results are promising, we believe there is significant room for improvement in how effectively we compute measures of feature collapse. There may be additional metrics that could help determine the optimal projection size of intermediate layers without the need to sweep over potential values. Furthermore, establishing a standard set of projection dimensions for common architectures would make our approach more accessible, enabling it to serve as a true drop-in replacement for existing models.

Given our promising results, we see several compelling directions for future work. In particular, our experiment on the effect of spectral normalization on neural collapse could have applications in supervised pretraining, where avoiding collapse is desirable to enable faster fine-tuning on downstream tasks. Additionally, we aim to explore how our model could better estimate the true effective depth of a network by iteratively placing skip connections, training the network for additional iterations, and then placing further skip connections. At some point, the network may stop shrinking, providing a potential method to measure the complexity of a problem by determining the minimum network depth required to solve it.

Given the importance of UQ in impactful applications, it’s crucial to ensure that our method and existing deterministic UQ approaches perform reliably across diverse problems. Further study is needed to validate their behavior and provide the public with trustworthy tools for informed decision-making.

References

- [1] Ido Ben-Shaul and Shai Dekel. Nearest class-center simplification through intermediate layers. In *Topological, Algebraic and Geometric Learning Workshops 2022*, pages 37–47. PMLR, 2022.
- [2] Tianqi Chen, Emily Fox, and Carlos Guestrin. Stochastic gradient hamiltonian monte carlo. In *International conference on machine learning*, pages 1683–1691. PMLR, 2014.

-
- [3] Erik Daxberger, Agustinus Kristiadi, Alexander Immer, Runa Eschenhagen, Matthias Bauer, and Philipp Hennig. Laplace redux—effortless Bayesian deep learning. In *NeurIPS*, 2021.
- [4] P Kingma Diederik. Adam: A method for stochastic optimization. (*No Title*), 2014.
- [5] Yarin Gal and Zoubin Ghahramani. Dropout as a bayesian approximation: Representing model uncertainty in deep learning. In *international conference on machine learning*, pages 1050–1059. PMLR, 2016.
- [6] Tomer Galanti, Liane Galanti, and Ido Ben-Shaul. On the implicit bias towards minimal depth of deep neural networks, 2022. URL <https://arxiv.org/abs/2202.09028>.
- [7] Tilmann Gneiting and Matthias Katzfuss. Probabilistic forecasting. *Annual Review of Statistics and Its Application*, 1(1):125–151, 1 2014. ISSN 2326-8298. doi: 10.1146/annurev-statistics-062713-085831. URL <http://www.annualreviews.org/doi/abs/10.1146/annurev-statistics-062713-085831>.
- [8] Tilmann Gneiting and Adrian E Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American statistical Association*, 102(477):359–378, 2007.
- [9] Felix Jimenez and Matthias Katzfuss. Vecchia gaussian process ensembles on internal representations of deep neural networks. *arXiv preprint arXiv:2305.17063*, 2023.
- [10] Tamara G Kolda and Brett W Bader. Tensor decompositions and applications. *SIAM review*, 51(3):455–500, 2009. Publisher: SIAM.
- [11] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. *Advances in neural information processing systems*, 30, 2017.
- [12] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [13] Jeremiah Liu, Zi Lin, Shreyas Padhy, Dustin Tran, Tania Bedrax Weiss, and Balaji Lakshminarayanan. Simple and principled uncertainty estimation with deterministic deep learning via distance awareness. *Advances in Neural Information Processing Systems*, 33:7498–7512, 2020.
- [14] Ilya Loshchilov and Frank Hutter. Sgdr: Stochastic gradient descent with warm restarts. *arXiv preprint arXiv:1608.03983*, 2016.
- [15] Wesley J Maddox, Pavel Izmailov, Timur Garipov, Dmitry P Vetrov, and Andrew Gordon Wilson. A simple baseline for bayesian uncertainty in deep learning. *Advances in neural information processing systems*, 32, 2019.
- [16] Jishnu Mukhoti, Andreas Kirsch, Joost van Amersfoort, Philip HS Torr, and Yarin Gal. Deep deterministic uncertainty: A new simple baseline. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 24384–24394, 2023.
- [17] Radford M Neal. *Bayesian learning for neural networks*, volume 118 of *Lecture Notes in Statistics*. Springer New York, NY, 1996.
- [18] Vardan Papayan, XY Han, and David L Donoho. Prevalence of neural collapse during the terminal phase of deep learning training. *Proceedings of the National Academy of Sciences*, 117(40):24652–24663, 2020.
- [19] Akshay Rangamani, Marius Lindegaard, Tomer Galanti, and Tomaso A Poggio. Feature learning in deep classifiers through intermediate neural collapse. In *International conference on machine learning*, pages 28729–28745. PMLR, 2023.
- [20] Hippolyt Ritter, Aleksandar Botev, and David Barber. A scalable laplace approximation for neural networks. In *6th International Conference on Learning Representations, ICLR 2018-Conference Track Proceedings*, volume 6. International Conference on Representation Learning, 2018.
- [21] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *3rd International Conference on Learning Representations*, pages 1–14, 2015.
- [22] Ledyard R Tucker. Implications of factor analysis of three-way matrices for measurement of change. *Problems in measuring change*, 15(122-137):3, 1963.
- [23] Joost van Amersfoort, Lewis Smith, Yee Whye Teh, and Yarin Gal. Uncertainty estimation using a single deep deterministic neural network. In *International conference on machine learning*, pages 9690–9700. PMLR, 2020.
- [24] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*, 2017.

Supplementary Material

Supplementary Material Contents:

A Method details	1
A.1 Measuring collapse	1
A.2 Projecting intermediate representations	1
A.2.1 Steps in projection algorithm	1
A.2.2 Computing channel-wise moments	2
A.3 Using PSCs	2
B Experiment details	2
B.1 Network Training	2
C Additional experiments and results	3
C.1 Sensitivity and Smoothness Example	3
C.2 DDU Case Study	4

A. Method details

This section provides details on our proposed methodology. The pseudo-code in Fig. 6 summarizes the steps necessary to augment a pretrained neural network with a PSC. The code between loading data and forming the prediction is detailed in the sections below.

A.1. Measuring collapse

Fig. 6 highlights that we choose the *candidate layer* using measures of neural collapse. In particular we measure $\mathcal{NC}1$ and $\mathcal{NC}4$ after each layer. This is different than the work of Rangamani et al. [19] where the metrics are measured before each layer. This difference arises due to our different motivations, in Rangamani et al. [19] the focus is on drawing parallels between intermediate feature collapse and feature collapse in the penultimate layer. However, in our work we focus on finding the layer that results in a feature vector with little collapse but high predictive performance. Computing both metrics can be done using simple modifications of the code from Rangamani et al. [19].

A.2. Projecting intermediate representations

For every call to the model we need to project the intermediate representations to get a feature vector. The steps to get this projection are shown in Algorithm 1. This algorithm assumes we have parameters $\Sigma, \mu, \Sigma_Z, \mu_Z$ in addition to A, B . The first set of these parameters correspond to the moments of X and to the moments of Z . The parameters A and B correspond to the factor matrices from the Tucker

```
# load model and data.
net = load_pretrained_net(...)
train, val, test = get_data(...)

# identify candidate layer.
candidate = measure_collapse(
    net, train, val
)

# fit projection.
projection = fit_projection(
    net,
    train,
    "tucker"
)

# fit PSC.
net = fit_psc(
    net=net,
    projection=projection,
    psc="ddu", # can replace with SNGP
    candidate=candidate,
    data=train
)

# predict.
y_hat, ood_score = net.predict(test)
```

Figure 6. Pseudo-Code for working with pretrained networks highlights the drop-in nature of PSCs.

decomposition of the channel-wise covariance of X .

All of these parameters must be learned from the data, and this is highlighted in the psuedo-code by the call to the function "fit_projection" which returns these parameters for future calls to Algorithm 1. Details on computing the moments are given in Appendix A.2.2. The details on the subroutines called in Algorithm 1 and computing A and B are given below.

Algorithm 1 Processing intermediate representations involves three steps: reshape, scale and project.

Input: $X, \Sigma, \mu, \Sigma_Z, \mu_Z, A, B$
 $X \leftarrow \text{reshape}(X)$ {E.g. Fig. 2 center panel.}
 $X \leftarrow \text{scale}(X, \Sigma, \mu)$
 $Z \leftarrow \text{project}(X, A, B)$ {Tucker.}
 $Z \leftarrow \text{scale}(Z, \Sigma_Z, \mu_Z)$ {Optional.}
Output: Z

A.2.1. STEPS IN PROJECTION ALGORITHM

Reshape The first operation involves reshaping the input. For the output of a single convolutional layer we use 1-

mode matricization [10] which simply turns inputs of shape $\mathbb{R}^{C \times h \times w}$ to shape $\mathbb{R}^{C \times hw}$ where we assume row-major flattening of each channel (i.e. $\text{vec}(\mathbf{A}^T)$ with $\mathbf{A} \in \mathbb{R}^{h \times w}$). If we want to combine multiple convolutional layers then we reshape each of them as describe and then concatenate them channel-wise.

Fig. 7 show that our approach for processing fully connect layers is similar and the resulting feature vector is also quite similar. By processing the inputs this way, the downstream methodology is identical regardless of the type of the candidate layer.

Scale The scaling operations assumes inputs of shape $\mathbf{X} \in \mathbb{R}^{C \times d}$ with $\boldsymbol{\mu} \in \mathbb{R}^{C \times d}$ and $\boldsymbol{\Sigma} \in \mathbb{R}^{C \times d \times d}$. Each input is then centered and scaled channelwise,

$$(\mathbf{X}_{i,:} - \boldsymbol{\mu}_{c,:}) \text{diag}(\boldsymbol{\Sigma}_{c,:,:})^{-1},$$

where $\text{diag}(\cdot)$ returns a diagonal matrix whose values match the diagonal of the input matrix, $\boldsymbol{\mu}_{c,:}$ represents the channel-wise mean, and $\boldsymbol{\Sigma}_{c,:,:}$ represents the channel-wise covariance.

Project When we perform the projection step we use the Tucker decomposition of the covariance matrix output from Algorithm 2 but with one exception. Since the output of the intermediate representation has been centered and scaled we first transform the covariance matrices before using the Tucker decomposition. More precisely, for channel c of the covariance tensor we compute the following,

$$\boldsymbol{\Sigma}_{c,:,:} = \text{diag}(\text{sqrt}(\boldsymbol{\Sigma}_{c,:,:}))^{-1} \boldsymbol{\Sigma}_{c,:,:} \text{diag}(\text{sqrt}(\boldsymbol{\Sigma}_{c,:,:}))^{-1},$$

where $\text{diag}(\text{sqrt}(\boldsymbol{\Sigma}_{c,:,:}))^{-1}$ denotes the diagonal matrix whose elements are the sample standard deviations of each dimension in the c^{th} channel. Once this transformation has taken place the projection step follows exactly as in Sec. 4.2.

A.2.2. COMPUTING CHANNEL-WISE MOMENTS

For dataset of N inputs we estimate "channelwise" means and covariances as illustrated in Algorithm 2. In words we use two passes through the data, one to estimate the mean of a hidden layer's output and then another pass to estimate the covariance of the same layer's output. We would like to make two comments on Algorithm 2:

(1) While in practice we vectorize the estimation of the moments (so the loop over channels and the loop over batches would be swapped), Algorithm 2 is written to emphasize that we do not consider the cross correlation between channels. We ignore cross channel correlation because if we did not, the size of resulting covariance matrix would be too large. For example with Resnet50 on CIFAR10 the covariance matrix of the candidate layer would be of

size $131,072 \times 131,072$. By ignoring cross channel covariance the project operation is greatly simplified and fast enough to be of practical use, even with large intermediate representations.

(2) While we use two passes through the training data (for each mean and covariance pair), we found that a subset of the full training set could be used to estimate the empirical moments without significant changes in results. However, it was important that the estimate of the mean was as accurate as possible before beginning to estimate the covariance. Using a streaming estimate of the mean along with a streaming estimate of the covariance gave us poor estimates of the feature covariance and this seemed to negatively impact the stability of training PSCs downstream.

Algorithm 2 Channelwise moments are estimated independently in batches.

Input: $\mathbf{X} \in \mathbb{R}^{N \times C \times d}$
 $\boldsymbol{\mu} \leftarrow \mathbf{0}, \boldsymbol{\Sigma} \leftarrow \mathbf{0}$
for c in $\{1, \dots, C\}$ **do**
 for b in B **do**
 $\boldsymbol{\mu}_{c,:} \leftarrow \frac{1}{|b|} \sum_{i \in b} \mathbf{X}_{i,c,:} - \boldsymbol{\mu}_{c,:}$
 end for
 for b in B **do**
 $\boldsymbol{\Sigma}_c \leftarrow \boldsymbol{\Sigma}_c + \sum_{i \in b} (\mathbf{X}_{i,c,:} - \boldsymbol{\mu}_{c,:})^T (\mathbf{X}_{i,c,:} - \boldsymbol{\mu}_{c,:})$
 end for
 $\boldsymbol{\Sigma}_c \leftarrow \boldsymbol{\Sigma}_c / N$
end for
Output: $\boldsymbol{\mu}, \boldsymbol{\Sigma}$

A.3. Using PSCs

Once the feature vector has been formed we need a model that can use the feature vector to make predictions. Following Liu et al. [13] we refer to this model as g and we require that g is *distance aware*. For this work we simply use a linear layer that is trained to map the projected intermediate representations, $\mathbf{Z}$ in Algorithm 1, to the class label. We then use a Laplace approximation for the posterior of the network weights. Where we further approximate the covariance of the Laplace approximation using the Kronecker factored Laplace approximation [20] as implemented in Daxberger et al. [3]. For the out-of-distribution prediction we can use quadratic discriminant analysis as documented in Sec. 5.2.

B. Experiment details

B.1. Network Training

For the neural networks in Sec. 5 we use the same training procedure and networks as in [16]. In short, we use stochastic gradient descent with momentum. The weight

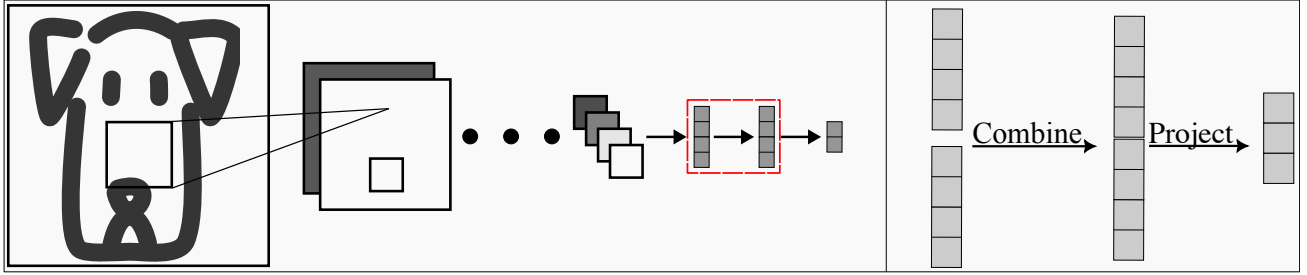


Figure 7. We can handle different layer outputs in a similar way to convolutional layers. The candidate layers of a network are both fully connected layers, and after concatenating and projecting the feature vector can be handled just as before.

decay, momentum and learning rates are set to the values in Appendix B.1. During training the learning rate was held constant and then multiplied by a factor of 0.1 at 150 and again at 250 epochs.

Hyperparameter	Value
Initial LR	0.1
Weight decay	5e-4
Momentum	0.9
Spectral Norm (if used)	3.0

Table 5. Standard hyperparameters were used during training and spectral norm was set to 3.0 if it was used.

The code for training and the networks (including spectral normalization) was an adaptation of the code from the DDU [16] paper, which is available on github (<https://github.com/omegafraagger/DDU>).

C. Additional experiments and results

In this section we present additional experimental results to supplement the work in Sec. 5.

C.1. Sensitivity and Smoothness Example

Borrowing from van Amersfoort et al. [23] we consider an example that demonstrates how the accuracy of the final prediction and the sensitivity of the model can be at odds. We additionally use this example to highlight that while spectral normalization helps ensure a smooth yet accurate classifier, as the penalty increases earlier layers become both sensitive and accurate.

In this example we consider $x_i \sim \mathcal{N}(0, \sigma^2)$ for $i = 1, 2$ and $y = \text{sign}(x_1) * \epsilon$, with ϵ denoting noise that flips the label on y with a small probability. We generate 3000 and 1000 samples for training and validation, respectively. We set $\sigma = 0.3$ and the probability of a flip set to 0.001. We then fit a neural network to predict the class label and we use a sequence of linear layers with Leaky-ReLU activations and

hidden dimensions of size $[4, 2, 2, 2, 2]$. Additionally, we use residual connections for all the layers of shape 2. The network is trained for 350 epochs using an Adam optimizer [4] with the learning rate adjusted from $3e-2$ to $3e-4$ using cosine annealing [14], additionally we set the weight-decay to $1e-5$.

During training we use the spectral normalization procedure from Mukhoti et al. [16]. We use train using two different values, one large and one small. After training we propagate the training data through the neural network and perform PCA on the intermediate representations. We then choose a training instance and apply two perturbations to it to generate two new psuedo data points. The first perturbation is *irrelevant* since we modify the input x along the second dimension which doesn't affect the class label. The second perturbation flips the sign and therefore changes the class label.

The results of this experiment are shown in Fig. 8. We see that for the smaller of the two spectral norm upper bounds that we can distinguish between the original input and the *irrelevant* perturbation even at the penultimate layer. With the larger upper bound the penultimate layer appears to have collapsed the points. However, with the larger upper bound we see that one of the earlier layers still separates the two points but is collapsed by the next layer. Furthermore, we see that all layers correctly place the *relevant* perturbation near the points of the opposite label. This shows that while the network seems to discard information necessary for sensitivity, spectral normalization ensures this information is preserved. If the spectral upper bound increases though, the layer where this information is preserved is earlier in the network. For deep networks, such as Resnet-50 in Sec. 5.1, the layer where this information is preserved is pushed earlier and earlier. Surprisingly, even without spectral normalization larger networks preserve this information at some earlier layer as demonstrated in the experiments in Sec. 5.

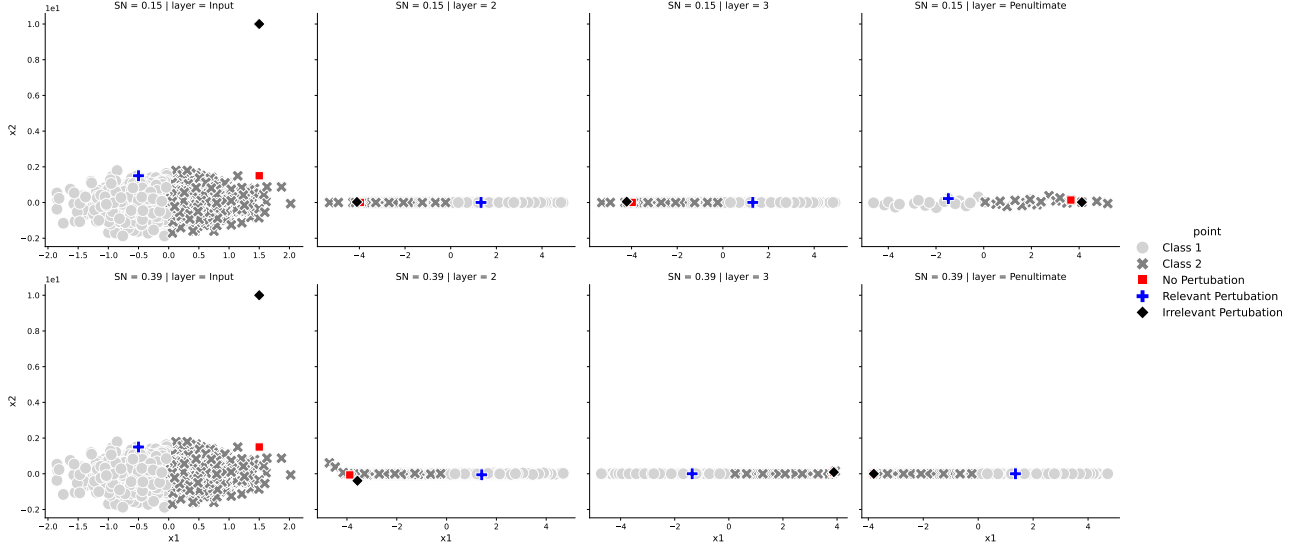


Figure 8. As the upper bound for the spectral norm increases unique points collapse at earlier layers, but are unique for some early layer. This figure shows the 2D intermediate representations after PCA for the sign-example from van Amersfoort et al. [23] as we change the upper bound for spectral normalization.

C.2. DDU Case Study

Predictive entropy histograms from Sec. 5.2 are shown in Fig. 9. In panel (b) we have the histograms for Resnet-18 and we see that the predictive entropy of the base model does well. Using spectral normalization doesn't improve on these results. Using the PSC pushes the entropy for iD-Ambiguous (A-MNIST) higher but also slightly increases the entropy for the iD-Clean (MNIST) examples. However, the result is we are able to separate the iD-Ambiguous from the iD-Clean with an AUROC that is similar to the base model. Combined with the feature densities of Fig. 4 we can distinguish all three data sources using the PSC.

For VGG-16 in panel (b) of Fig. 9 we see a similar story as we did with Resnet-18. Using the PSC increases the predictive entropy of the iD-Ambiguous instances but also slightly increases the entropy of the iD-Clean instances. However, the AUROC for using the PSC predictive entropy to separate iD-Clean and iD-Ambiguous is higher in this case. Just as before, we can combine these results with the feature density results to cleanly separate all three data sources using the PSC.

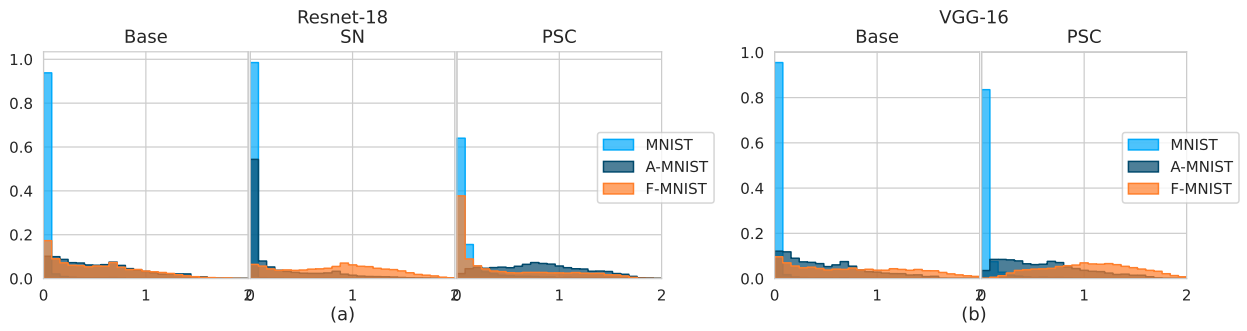


Figure 9. Predictive entropy can be used to separate iD-Ambiguous and iD-Clean. The figure shows the predictive entropy values when the base model used was Resnet-18 (a) or VGG-16 (b).